

ESP32 + NRF24 Real-Time Audio Streaming Architecture

Overview

This project implements a real-time wireless audio link between two ESP32 DevKit V1 boards using:

- INMP441 I2S microphone
- NRF24L01+ PA/LNA radio modules
- ESP32 internal DAC (GPIO25)
- LM386 audio amplifier

The system continuously captures audio on the transmitter, packetizes it, sends it over the NRF24 link, and immediately reconstructs and plays the audio on the receiver.

The implementation prioritizes:

- Low latency
- Simplicity
- Real-time streaming
- Minimal RAM usage

No audio compression is currently used. Raw 8-bit audio samples are transmitted directly.

Hardware

Transmitter

INMP441 Connections

INMP441	ESP32
VDD	3.3V
GND	GND
L/R	GND
SCK	GPIO13
WS	GPIO16
SD	GPIO17

NRF24 Connections

NRF24	ESP32
CE	GPIO4
CSN	GPIO5
MOSI	SPI MOSI
MISO	SPI MISO
SCK	SPI SCK

Receiver

NRF24 Connections

Same as transmitter.

Audio Output

ESP32	Function
GPIO25	DAC output

GPIO25 connects to the LM386 amplifier input through a resistor divider.

Audio Pipeline

Transmitter Side

Audio flow:

INMP441 → I2S → DSP Processing → Packet Builder → NRF24

Step 1: Audio Capture

The INMP441 microphone produces 24-bit audio samples.

The ESP32 receives these through I2S.

Configuration:

- 8000 Hz sample rate
- Mono
- 32-bit I2S transfers
- Left channel only

The microphone continuously fills DMA buffers.

Step 2: DC Offset Removal

INMP441 microphones usually contain a DC bias.

An exponential moving average is used:

$$\text{dc_offset} = (\text{dc_offset} * 63 + \text{sample}) / 64$$

Each sample is centered:

$$\text{centered} = \text{sample} - \text{dc_offset}$$

This prevents the audio waveform from drifting away from zero.

Step 3: Gain

The centered microphone signal is amplified in software.

Example:

$$\text{scaled} = (\text{centered} * \text{GAIN}) \gg 16$$

Typical gain values:

- 16
- 24
- 32

Higher gain increases volume but also increases noise and clipping risk.

Step 4: Convert to 8-Bit Audio

The ESP32 DAC accepts values from:

0 to 255

with:

128 = silence

The microphone signal is converted:

value = 128 + scaled

Then clamped:

$0 \leq \text{value} \leq 255$

This produces a stream of unsigned 8-bit audio samples.

Step 5: Packet Formation

NRF24 packets are fixed at 32 bytes.

Packet layout:

Byte 0:

- Sequence number

Bytes 1-31:

- Audio samples

Structure:

[SEQ][AUDIO][AUDIO][AUDIO]...[AUDIO]

This allows:

31 audio samples per packet

At 8000 Hz:

8000 samples/sec

$8000 / 31 \approx 258$ packets/sec

Step 6: Transmission

Each packet is sent immediately:

```
radio.write(packet, 32)
```

The transmitter continuously streams packets.

No buffering of complete recordings is performed.

This creates a live audio stream.

Receiver Side

Audio flow:

NRF24 → Ring Buffer → DAC → LM386 → Speaker

Step 1: Receive Packets

The receiver constantly polls:

```
radio.available()
```

When a packet arrives:

```
radio.read(packet, 32)
```

The sequence byte is ignored unless packet-loss detection is desired.

Step 2: Push Audio Into Ring Buffer

Each packet contains:

31 audio samples

These samples are copied into a circular buffer.

Example:

```
packet[1] packet[2] ... packet[31]
```

↓

ring buffer

The ring buffer absorbs small timing variations in radio delivery.

Step 3: DAC Playback

A dedicated playback task runs at:

8000 Hz

Each cycle:

1. Pull one sample from ring buffer
2. Output sample to DAC GPIO25

Example:

```
dac_output_voltage(DAC_CHANNEL_1, sample);
```

If the ring buffer is empty:

```
dac_output_voltage(DAC_CHANNEL_1, 128);
```

which outputs silence.

Ring Buffer Design

A ring buffer is used to separate:

Radio timing

from

Audio playback timing

Without the ring buffer:

Radio packet jitter would directly create audio glitches.

The ring buffer acts as a small reservoir of audio samples.

FreeRTOS Tasks

Transmitter

micTask

Responsibilities:

- Read I2S microphone
- Remove DC offset
- Apply gain
- Convert to 8-bit samples
- Fill NRF packets
- Transmit packets

Runs on:

Core 0

Receiver

nrfTask

Responsibilities:

- Receive packets
- Extract audio samples
- Push samples into ring buffer

Runs on:

Core 0

dacTask

Responsibilities:

- Maintain exact playback rate
- Read ring buffer
- Feed DAC

Runs on:

Core 1

Keeping playback isolated improves timing stability.

Audio Characteristics

Current format:

- 8000 Hz sample rate
- Mono
- 8-bit unsigned PCM

Bandwidth:

$8000 \text{ samples/sec} \times 8 \text{ bits}$

$\approx 64 \text{ kbps audio}$

This is suitable for:

- Voice
- Walkie-talkie style communication
- Intercom applications

Not suitable for:

- Music
 - High-fidelity audio
-

Known Limitations

Packet Loss

Current implementation does not recover lost packets.

If a packet is missed:

31 audio samples are lost.

This produces a brief click or dropout.

No Compression

Every audio sample consumes one byte.

Future versions may use:

- IMA ADPCM
- μ -law
- Delta encoding

to increase range and reliability.

NRF24 Throughput Limits

The design intentionally uses:

- 8000 Hz audio
- 1 Mbps radio mode

to remain within the practical throughput limits of NRF24 modules.

Higher audio rates may increase latency and packet loss.

Future Improvements

Potential upgrades:

1. Sequence number tracking
 2. Packet-loss concealment
 3. Voice activity detection
 4. Audio compression (ADPCM)
 5. Dynamic gain control
 6. Noise gate
 7. Low-pass filtering
 8. Push-to-talk support
 9. Multiple-node audio routing
 10. Encrypted audio packets
-

Summary

The system implements a simple real-time wireless audio stream:

1. INMP441 captures audio.
2. ESP32 converts microphone data to 8-bit PCM.
3. Samples are packed into 32-byte NRF24 packets.
4. NRF24 transmits packets continuously.
5. Receiver stores samples in a ring buffer.
6. DAC outputs audio through GPIO25.
7. LM386 amplifies the signal for a speaker.

The architecture is intentionally simple and serves as a solid base for integrating voice communication into larger ESP32 projects.